

AI Agent Knowledge Base for a Financial Company

Status: Draft (pending approval) · Date: 2026-08-27

Contents

1. [The Problem Nobody Wants to Own](#)
2. [The Temptation of Shiny Infrastructure](#)
3. [Where Do Three Million Files Live?](#)
4. [Teaching the System to Read](#)
5. [The Art of Breaking Documents Apart](#)
6. [Three Ways to Find a Needle](#)
7. [The Web Between the Documents](#)
8. [What Was True Last March?](#)
9. [The Corpus Never Sits Still](#)
10. [Trust, but Verify — Every Night](#)
11. [Answers You Can Take to a Regulator](#)
12. [The Day Everything Goes Wrong](#)
13. [How Big Is This, Really?](#)
14. [The Road from Here](#)
15. [The Shape of the Thing](#)

Appendices: [A — Requirements Reference](#) · [B — Technology Choices](#) · [C — Risk Register](#)

The Problem Nobody Wants to Own

Somewhere in the firm's file shares, SharePoint sites, mail archives, and document management systems sit roughly **three million documents**: credit agreements, amendments, side letters, board decks, risk memos, spreadsheets, and thirty years of scanned paper. Somewhere in there is the answer to almost any question a lawyer, credit officer, or risk analyst might ask.

Nobody can find it.

Keyword search returns four hundred hits. The person who knew which document mattered retired. The version someone finally digs up was superseded two amendments ago — and they won't discover that until the deal goes wrong.

So the ask sounds simple: *build an AI assistant that answers questions from our documents*. But this is a regulated financial firm, so the real ask is longer:

- Answer from the documents, and **prove it** — with citations down to the page, tied to the exact version used.

- Show a user **only what they're entitled to see**. Not one chunk more. Ethical walls between deal teams are law, not etiquette.
- Keep everything — documents, search indexes, models, logs — **inside the firm's walls**.
- Keep a **tamper-evident record** of every question and answer, because a regulator may ask for it years from now.
- And when the system doesn't know, it must **say so** instead of inventing something plausible.

That last point deserves emphasis. A chatbot that hallucinates a covenant threshold isn't a productivity tool; it's a liability engine. Everything in this design flows from taking these constraints seriously.

Version 1 is deliberately modest in one way: it is **read-only**. It answers questions. It executes no trades, sends no emails, makes no decisions. Walk before you run.

The Temptation of Shiny Infrastructure

The first challenge isn't technical — it's restraint.

The standard architecture for a system like this, circa any vendor pitch, involves a vector database, a search cluster, a message broker, a workflow engine, and a graph database: five distributed systems to operate, patch, secure, and keep synchronized. Every pair of systems is a place where data can disagree. Every extra cluster is an on-call rotation.

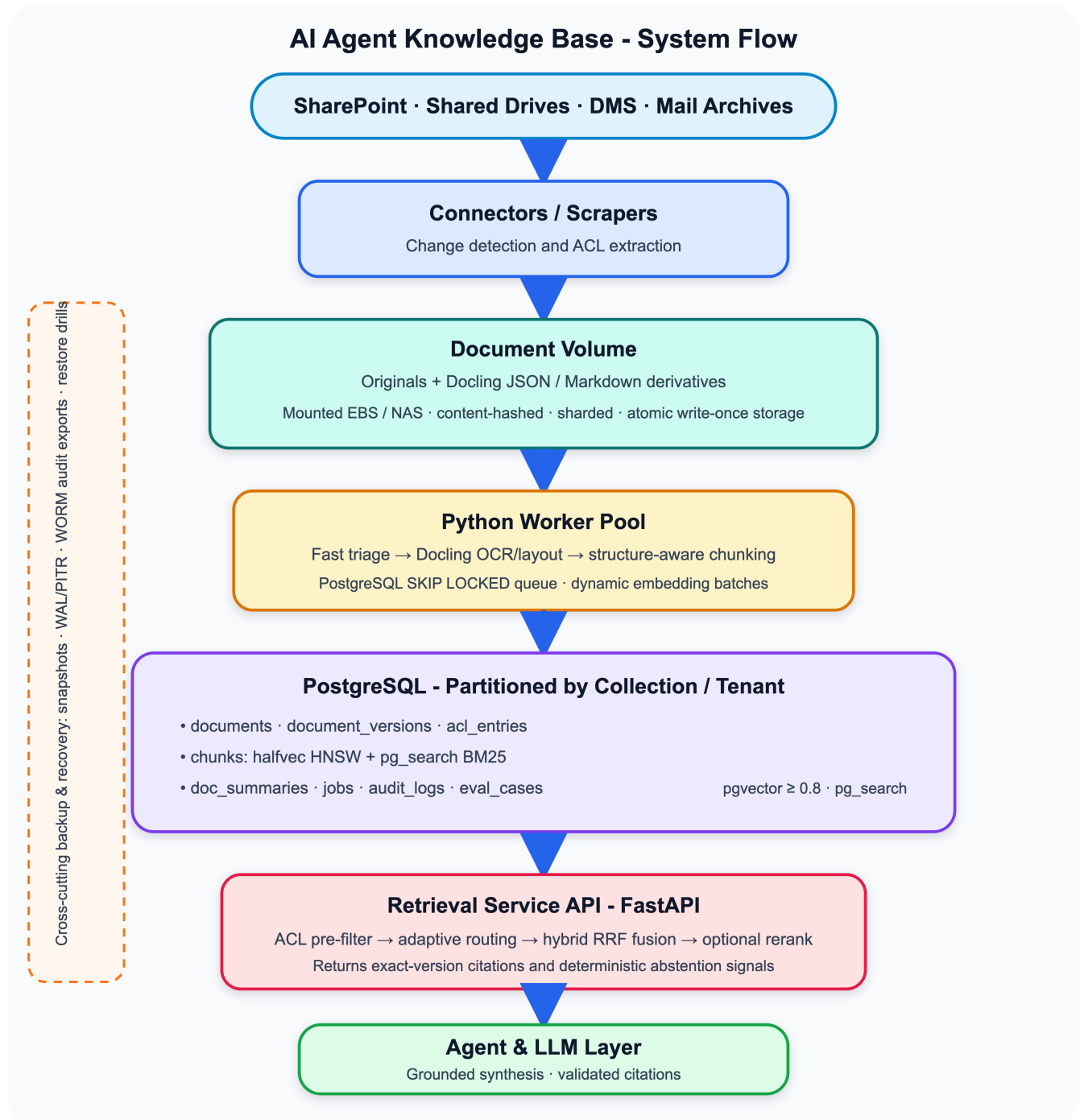
Complexity like that doesn't just cost money — it can bring a project to its knees. Every additional system needs specialists to run it, integration code to connect it, and a seat at the table when something breaks at 2 a.m. Projects don't usually die from missing features; they die from drowning in their own plumbing before the first user ever asks a question.

So this design adopts **simplicity as its guiding principle**: the simplest, most elegant architecture that still delivers the full functionality — the minimum number of moving parts, chosen deliberately, not by accident. Simplicity is what makes the project *do-able*. It makes it flexible (fewer parts to rearrange when requirements shift), maintainable (fewer things that can break, fewer experts required), and affordable (no fleet of clusters, no heavy hardware). We can prototype fast and put something real in front of users in weeks. We don't need a big team to build it — or to keep it alive afterward.

Here's what makes that principle realistic rather than wishful: our corpus — big as it feels — produces about **20–25 million searchable chunks**. That is not "big data." That fits in one well-fed PostgreSQL instance with room to spare.

So the solution is a bet on boring technology: one PostgreSQL cluster holds everything transactional — the document registry, the entitlements, the search indexes (vector *and* keyword), the job queue, and the audit log. The team already knows how to run PostgreSQL. Access control becomes a SQL join instead of a cross-system synchronization problem. There is exactly one source of truth.

And because bets should be falsifiable, the design writes down **scaling gates** — measurable thresholds (chunk count above 50 million, p95 latency above 2 seconds despite tuning, index churn degrading reads) at which we would graduate to specialized infrastructure. Until a gate trips, we don't. When one does, the migration is a background re-index, not a crisis — for reasons explained below.



Where Do Three Million Files Live?

Databases are terrible places for file blobs, and cloud object storage is off the table — the compliance boundary says everything stays on infrastructure the firm controls. So originals live on **mounted volumes** (AWS EBS behind an NFS export, or the on-prem NAS), and the database stores only a `storage_uri` pointing at each file.

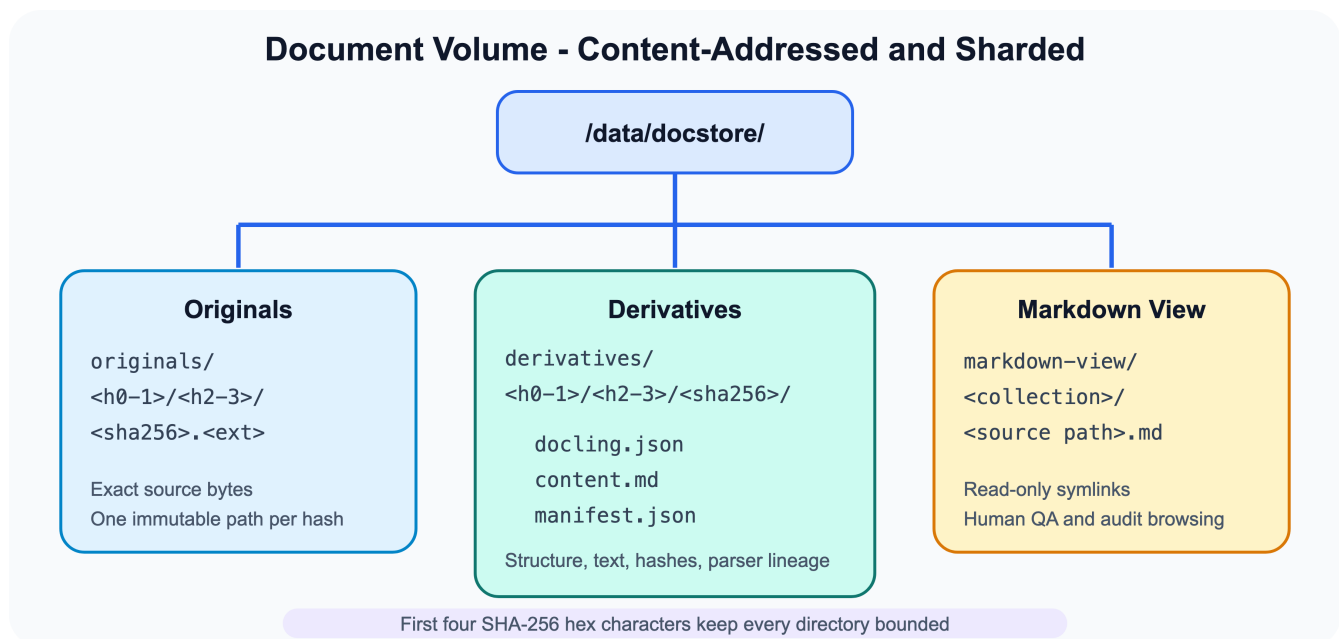
That sounds mundane until you ask what three million files do to a filesystem. Dump them in one directory and every listing crawls. Let people overwrite files in place and you can never again prove what a citation pointed to.

Two disciplines solve this:

- **Content addressing.** Every file is stored under its SHA-256 hash, sharded into subdirectories by the first four hex characters (`originals/ab/cd/<sha256>.pdf`) so no directory ever holds more than a few thousand entries. The same bytes always land at the same path; a changed document produces a *new* hash and a *new* path.
- **Write-once.** Workers write to a temp path, `fsync`, and atomically `rename()` into place. Nothing is ever overwritten. A file, once written, is immutable — optionally enforced with `chattr +i` so even a root process can't quietly edit history.

This buys something subtle and valuable: **a snapshot of the volume is consistent by construction**, at any instant, with no quiescing — a half-written file exists only under a temp path that nothing references. Remember that when we get to "The Day Everything Goes Wrong."

Alongside each original live its **derivatives**: the parsed, normalized representations (structured JSON plus clean Markdown). A weekly scrub job re-hashes samples and compares against the database, hunting silent corruption.



Capacity is comfortable: ~3 TB of originals, 1–2 TB of derivatives; provision 8–10 TB and grow with LVM.

Teaching the System to Read

Challenge: the corpus is a museum of formats. Born-digital PDFs, Word contracts, PowerPoint decks where the story hides in speaker notes, spreadsheets, and scans of paper signed before some analysts were born.

Running everything through a heavyweight OCR/layout pipeline would take months. Running everything through a fast text extractor would butcher the scans and mangle every table.

The solution is triage. A fast path (PyMuPDF) handles born-digital files at 50–100 pages per second per node. A heavy path — GPU workers running Docling with layout analysis and OCR — handles scans, complex multi-column layouts, and decks at 8–15 pages per second. Every scanned page gets an **OCR confidence score** that follows its text all the way into answers: a citation built on shaky OCR says so.

Two principles govern the pipeline:

Parse once, index many times. Parsing three million files takes weeks of compute; we never want to do it twice. The parsed derivatives are stored permanently. Re-chunking, re-embedding, upgrading models, even migrating to a different search engine — all read from derivatives, never from raw files. This is what makes the scaling gates cheap to cross and the whole search layer *disposable*.

Deduplicate before indexing. The same memo lives in five SharePoint sites and eleven inboxes. Exact duplicates are caught by hash before they're even fetched twice; near-duplicates by MinHash/SimHash on normalized text. Aliases map every copy to one canonical version. Raw estimate: ~45 million potential chunks collapse to 20–25 million indexed ones — half the index we'd otherwise pay for.

The whole pipeline runs as idempotent steps in a transactional job queue — plain PostgreSQL rows claimed with `SKIP LOCKED`, no message broker. A crashed worker resumes exactly where it stopped; a re-delivered event is a no-op.

The Art of Breaking Documents Apart

Search engines don't retrieve documents; they retrieve *chunks*. Chunk badly and everything downstream suffers.

Split a contract mid-sentence at every 500 tokens and you get fragments that match queries but can't support answers. Treat a financial table as prose and its meaning evaporates — a number without its header row is noise.

The solution: cut along the document's own seams.

- **Narrative documents** split at heading boundaries, 400–800 tokens, on clean paragraph breaks. Each chunk carries its full heading path — `Credit Agreement > Section 4.2 > Representations` — so it never forgets where it came from.
- **Slides:** one slide per chunk, with the deck title, slide title, and presenter notes together (in decks, the notes are usually where the truth lives).
- **Tables:** kept whole, or split as row-groups that each repeat the header row; the raw structured version rides along in metadata for exact numeric lookups.
- **Scans:** chunked like narrative, with their OCR confidence attached; anything below 85% is flagged in prompts and citations.

One principle rules the data model: **documents are not chunks**. A document has lifecycle, lineage, entitlements, and legal retention. A chunk is a disposable search artifact, rebuildable at any time from derivatives. Confusing the two is how systems end up unable to delete what the law requires them to delete.

Three Ways to Find a Needle

Now the heart of the matter. Users ask two fundamentally different kinds of questions, and each breaks the search method built for the other:

- *"What is our aggregate exposure to commercial real estate in Europe?"* — thematic, conceptual. Keyword search is useless; no document contains that sentence.
- *"Find clause 7.3(b) of the Meridian facility agreement."* — an exact needle. Semantic search is actively dangerous here; it happily returns something *similar* to clause 7.3(b), which is worse than nothing.

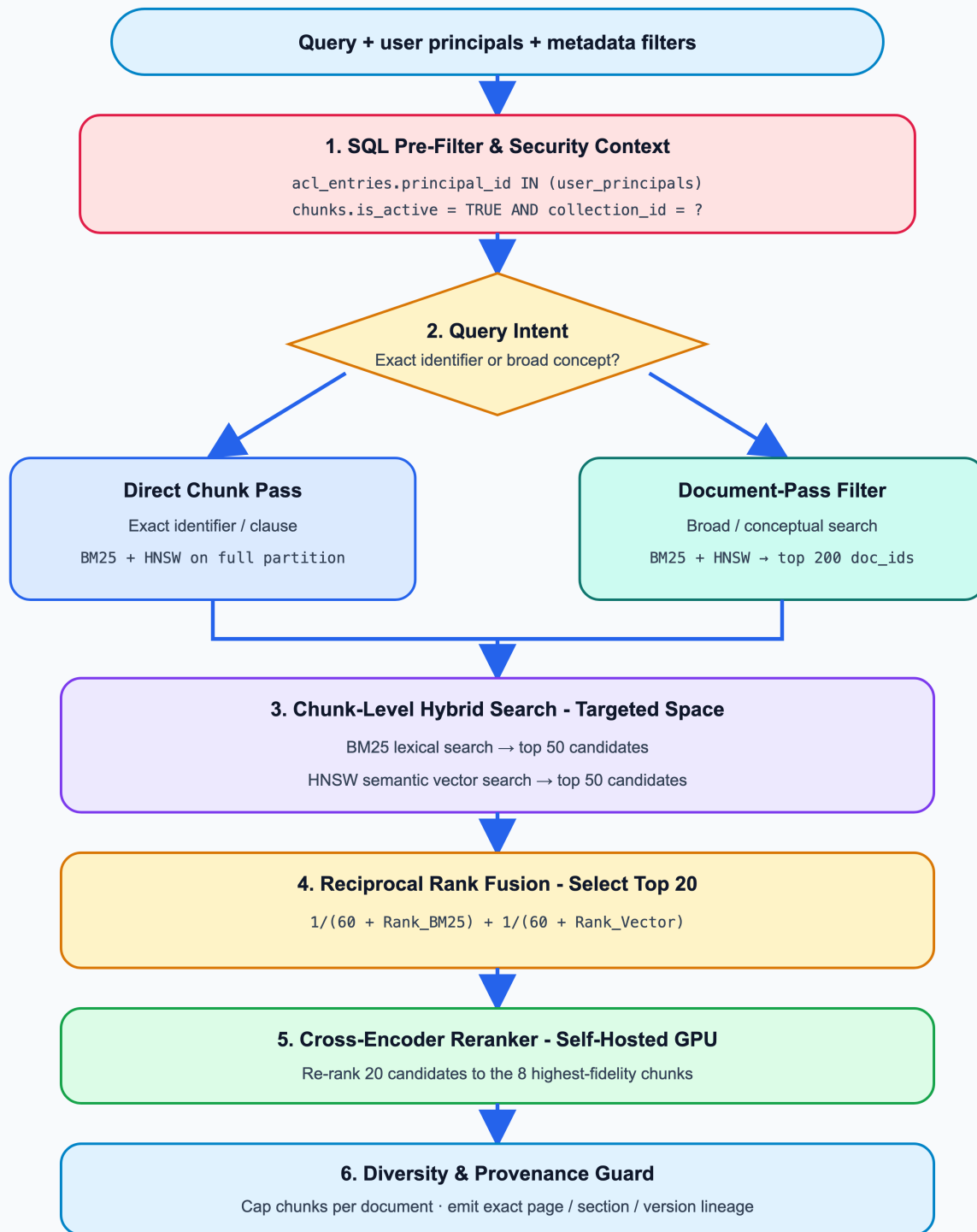
Solution one: run both searches, always. Semantic search (vector embeddings, computed by self-hosted models on our GPUs — nothing leaves the boundary) catches meaning and paraphrase. Keyword search (BM25 full-text) catches tickers, ISINs, clause numbers, party names. Their results merge by reciprocal rank fusion, and a cross-encoder reranker on GPU picks the best handful for the answer.

Solution two: search at two altitudes. Millions of chunks is a noisy haystack for a broad question. So every document also gets a one-paragraph **summary with its own embedding** — a 3-million-entry "card catalog." Broad queries hit the catalog first, pick the promising documents, then search chunks only within them. Needle queries — anything with an exact identifier — skip the catalog and hit the full chunk index directly, because a summary might not mention clause 7.3(b), and a missed clause is a failed audit.

Solution three: after matching, widen the lens. Chunks are sized for *matching*, but the sentence that matches a query and the sentence that answers it are often neighbors. So each selected chunk is expanded to its parent section — its siblings by heading path, within a token budget — before the model sees it. The match finds the spot; the section provides the meaning.

And beneath every one of these paths, without exception, sits the entitlement check. **ACL filtering happens in SQL, before ranking.** An unauthorized document doesn't rank low — it doesn't exist. The permission-leak tolerance in this design is written as a number: **0.00%**.

Adaptive Hybrid Retrieval



The Web Between the Documents

A few weeks into any project like this, someone asks the question that vectors and keywords cannot answer:
"Which amendments modify this credit agreement?"

Similarity search finds text that *resembles* other text. But an amendment doesn't resemble the agreement it modifies — it references it, tersely, by title and date. The relationships between documents — supersession, amendment, exhibit, citation, same deal — are the corpus's skeleton, and text search is blind to it.

Here's the lucky part: legal documents *announce* their relationships. "This Amendment No. 3 to the Credit Agreement dated March 15, 2024..." is practically a database row wearing a costume. No AI required — regexes and parsers extract these cross-references, along with entities (parties, ISINs, deal names), during ingestion.

So the design adds a document graph, in plain PostgreSQL tables: typed edges (`amends`, `supersedes`, `exhibit_of`, `references_clause`, `cites`) between documents, plus an entity index recording who is mentioned where. Each edge records how it was extracted and with what confidence.

The graph earns its keep twice:

At retrieval time, after the hybrid search returns its chunks, the engine walks one hop: retrieve a credit agreement, and its amendments come along — even though they never matched the query. Retrieve something that has a `supersedes` edge pointing at it, and the answer is *flagged as superseded* — structurally, not by hoping the newer version happened to rank. Every hop joins the ACL table first: a link must never reveal so much as the existence of a document the user can't see.

For humans, the graph renders as an **interconnected wiki**: one Markdown file per document, with YAML frontmatter (type, dates, classification, status) and `[[wikilinks]]` for every relationship — browsable in Obsidian, greppable with ripgrep, organized in a readable tree (`wiki/documents/<collection>/<type>/<year>/...`, plus a stub page per entity listing everything it appears in). Compliance officers and domain experts navigate the corpus by its actual structure instead of playing search-term roulette.

One rule keeps the wiki honest: **it is a projection, not a source**. Nobody edits wiki files. They are rendered nightly from the database and the stored derivatives, and can be deleted and regenerated at any time without losing a byte of truth. And because files on disk have no row-level security, the full wiki renders only inside the curators' enclave — links leak existence, and existence is confidential too.

Later (v2), community detection over this graph will cluster the corpus into themes with summary pages — the corpus explaining its own neighborhoods.

What Was True Last March?

A question that will absolutely be asked: "*What was the covenant threshold as of Q3 2024?*"

A system that only knows "current vs. superseded" answers with today's amendment — fluently, confidently, and wrongly. In finance, *when something was true* is half the fact.

The solution is to give time a first-class seat. Every document version carries an effective window (`effective_from` / `effective_to`); every graph edge carries a validity window. These come from the documents themselves — effective-date clauses are dated boilerplate, extractable deterministically — and from lineage: when a supersession edge is created, it closes the predecessor's window automatically.

On top of that:

- The retrieval API accepts an **as_of_date**. Instead of filtering to current versions, it filters to versions *in force on that date* — and the answer is explicitly flagged as historical.
- When retrieved passages disagree *because they're from different eras*, the system doesn't report a contradiction; it reports a **timeline**: "the threshold was X from March 2024, amended to Y effective November 2024."
- Ranking gets a temporal sense: queries with date intent boost chunks near that date; queries without it get a mild boost toward in-force versions, so stale text can't outrank live text on similarity alone.

The Corpus Never Sits Still

Everything so far described a system at rest. Real corpora churn daily: new documents, revised versions, deletions, and — most dangerous of all — entitlement changes.

Each event must propagate through every layer we've built: registry, chunks, vectors, indexes, summaries, graph edges, wiki pages. Miss one and the layers start lying to each other.

The solution is a defined lifecycle with three hard rules.

When a document is **added**, it flows through the standard pipeline and is searchable within minutes. When one is **updated**, the new version gets new chunks and a **supersedes** edge; the old version's chunks are *deactivated, never deleted* — that's rule one: **soft-deactivation**, because an audit-log citation from last year must still resolve to the exact text the user saw, for as long as the records policy says. When a document is **removed** at the source, it's tombstoned: invisible to retrieval the instant the transaction commits (the active-flag and ACL filters run before ranking, so no index rebuild is needed), while the underlying files ride out their retention period.

Rule two: **the visibility flip is atomic**. Old chunks off, new chunks on, current-flag moved — one transaction. No query ever sees a document half-updated.

Rule three: **ACL changes outrank everything**. A revoked entitlement propagates within **five minutes**, monitored as a first-class alert, with a daily full re-sync sweeping up anything a missed webhook dropped. Stale content is embarrassing; stale permissions are a breach.

The slower layers refresh on their own cadence — wiki re-renders and link re-extraction nightly, cluster refresh and physical garbage collection weekly to monthly. And because deactivated vectors accumulate as dead weight in the vector index, a weekly recall check compares the index against a brute-force scan on a sample; if recall sags, that partition gets rebuilt in a maintenance window.

Trust, but Verify — Every Night

Two uncomfortable truths about RAG systems: they degrade quietly, and they hallucinate confidently. A connector fails silently on a Tuesday; nobody notices until someone asks about a document ingested in the gap. An embedding model update shifts rankings; recall drops three points with no error message anywhere.

The solution is to treat evaluation like a nightly deployment gate, not a launch-week ritual.

The yardstick is a **golden test suite**: 300+ real questions curated with Legal, Risk, Credit, and Operations, with known correct answers *and known correct citations*. It includes as-of temporal cases, adversarial permission-leak attempts, and — crucially — an **unanswerable set**: questions whose answers are deliberately absent, or locked behind entitlements, or only in superseded text. The correct response to those is abstention. A confident answer to an unanswerable question is a hallucination, caught in the act.

Every night, after the day's sync settles, the full suite runs against **production** indexes:

- Results compare against a rolling 7-day baseline. Recall drops more than 2 points? Citation precision slips? Abstention correctness falls? **Any** permission leak? The gate trips: on-call is paged and ingestion freezes until a human understands why. A failed eval is a failed deploy.
- A **reconciliation count** runs across three layers — source systems → registry → active chunks — so a silently failed connector shows up as a number that doesn't match, tonight, not at quarter-end.
- A **self-retrieval probe** samples documents added or changed that day, generates a query from each one's own content, and verifies the document comes back. Proof that new content is *findable*, not merely stored.

Hallucination gets its own continuous watch, because golden questions can't cover real traffic. A **self-hosted judge model** (nothing leaves the boundary) samples production answers daily and checks entailment — does each cited chunk actually support the claim it's attached to? The groundedness rate is trended and alerted. Abstention rates are watched in both directions: a sudden drop means the system got brave about things it shouldn't answer; a spike means retrieval degraded. And monthly, humans re-score a sample of the judge's verdicts, so the automated number stays worth reporting to Compliance.

Answers You Can Take to a Regulator

The generation step — the part everyone calls "the AI" — is deliberately the most constrained component in the system.

The model receives only the top-ranked, entitlement-filtered, section-expanded chunks, each wearing a header: document, page, version. The prompting contract is strict:

- **Every factual claim carries a citation token**, validated by middleware against the chunks actually retrieved. An answer with an unverifiable citation is rejected before the user sees it. The model cannot cite what it wasn't given.
- **Contradictions are surfaced, not smoothed over** — and when versions differ across time, presented as a timeline ("What Was True Last March?").
- **Insufficient evidence gets an honest refusal**: *"The available documentation does not contain sufficient evidence to answer this inquiry."* The system is graded on saying this at the right times — that's what the unanswerable set is for.

And everything is written down. Every query logs the user, their evaluated roles, the filters applied, every chunk ID retrieved with its scores, the model and prompt versions, the generated answer, and the rendered citations. Monthly partitions of this log are exported nightly to **write-once storage** with hash manifests — the export includes the chunk *text* itself, so the record remains self-contained even after some future re-chunking retires

the IDs. Deletion happens through the records-management process on the retention schedule. Not through engineering. Ever.

The Day Everything Goes Wrong

Now the chapter nobody enjoys writing. Suppose the worst: a bad ACL sync poisons entitlements, an operator fat-fingers a `DROP TABLE`, ransomware hits, an availability zone burns. What survives?

First, an unpopular truth: **replicas are not backups**. A standby replica faithfully replays your `DROP TABLE` within milliseconds. Replication is for hardware failure; backups are for mistakes and malice.

The database ships every write to an archive continuously (point-in-time recovery to any second — losing at most an hour is the written objective) plus weekly full and daily incremental base backups. Backup storage lives in a **separate account with independent credentials**, so no single compromised admin can destroy both the data and its safety net. Volume snapshots — consistent by construction, thanks to the write-once discipline — are copied cross-AZ and **locked** against deletion, so even ransomware with admin keys can't erase the past. The audit archive is write-once by design.

Restores follow a scripted order, because the database and the volume must agree: **database first** to the chosen moment, **volume snapshot from at-or-after** that moment (write-once means a newer snapshot only has harmless extras; an older one might be missing referenced files — newer is always safe, older never is), then a reconciliation pass that re-hashes every referenced file and queues re-fetches for gaps. **ACLs re-sync before the API reopens** — restoring stale entitlements would be a permission leak with a restore-runbook as the root cause.

And because a backup that's never been restored is a rumor: **a monthly automated restore test** rebuilds the database from backups in an isolated environment, verifies integrity, and runs the golden suite against it — with a failed restore paging on-call like a production outage. Quarterly, a full disaster-recovery drill runs the entire sequence against the clock, and the results go to Compliance and Business Continuity in writing.

Recovery targets, stated plainly: lose at most one hour of data; be back in service within eight for a full-site event, faster for lesser failures.

How Big Is This, Really?

The numbers that make the single-database bet rational:

On the document volume:

What	Size
Originals (3M × ~1 MB)	~3 TB
Parsed derivatives (JSON + Markdown)	~1–2 TB
Rendered wiki tree	~0.3–0.5 TB
Headroom for growth	~2–3 TB

What	Size
Provisioned	8–10 TB

In PostgreSQL:

What	Size
Chunk text (20–25M chunks)	~40–60 GB
Vectors + HNSW index (half-precision)	~50 GB
Full-text (BM25/GIN) indexes	~20–30 GB
Registry, ACLs, summaries	~15–25 GB
Document graph	~5–10 GB
Audit log growth	~2–5 GB/month
Steady state	~200–300 GB

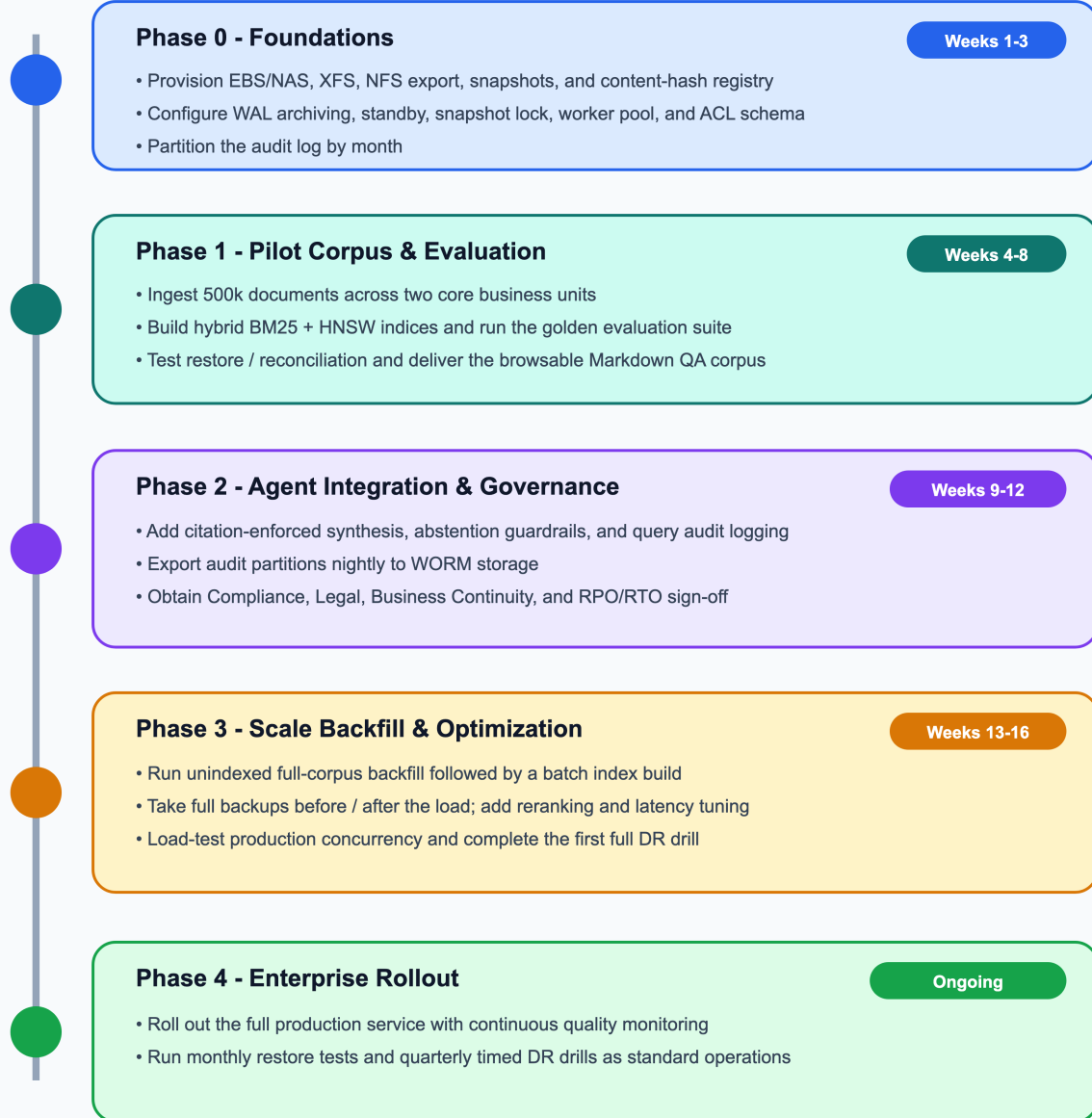
A single instance with 128–256 GB of RAM keeps the vectors and hot indexes in memory. Backups run 2–3× database size; the first volume snapshot equals used bytes, with small daily deltas after.

Retrieval targets: **p95 under 1.5 seconds** for search, under 4.5 end-to-end. Ingestion: full backfill in weeks (bulk-loaded into unindexed tables, with indexes built once at the end — building indexes during a 25-million-row load is how backfills become quarter-long projects), then incremental sync in minutes thereafter.

The Road from Here

The build order is chosen so that trust is earned before scale is attempted:

Delivery Roadmap



Phase 0 (weeks 1–3) lays foundations: storage, database, WAL archiving, ACL schema. **Phase 1 (weeks 4–8)** ingests a 500k-document pilot across two business units, builds the hybrid indexes, extracts links and effective dates, renders the first wiki — and runs the golden suite for the first time, while curators evaluate extraction quality on the same corpus they QA. **Phase 2 (weeks 9–12)** adds the guarded generation layer — citations, abstention, audit logging, WORM export — plus graph expansion, as-of retrieval, and the nightly evaluation gate, ending in Compliance/Legal/BC sign-off. **Phase 3 (weeks 13–16)** is the full backfill and load testing, bracketed by base backups and closed with the first timed DR drill. **Phase 4** is enterprise rollout — at which point the nightly gates, restore tests, and drills stop being milestones and become weather.

v2 candidates wait behind evaluation evidence, in the spirit of the opening bet: LLM-based relationship extraction on high-value collections, thematic clustering with summary pages, and any migration past a scaling gate.

The Shape of the Thing

Strip away the details and the design is six decisions:

1. **Simplicity above all.** The fewest moving parts that still deliver the functionality. Simplicity is what makes the project do-able, flexible, maintainable, and affordable — buildable by a small team, prototyped in weeks, run without a fleet of clusters or heavy hardware. Complexity is treated as the project's primary risk, admitted only through measured gates.
2. **One system of record.** PostgreSQL holds truth — content indexes, entitlements, graph, queue, audit — so consistency is a transaction, not a distributed-systems project.
3. **Parse once; everything downstream is disposable.** Originals and derivatives are permanent; chunks, vectors, indexes, and wiki are projections, rebuildable at will. This is what makes every future migration boring.
4. **Search three ways** — meaning, keywords, structure — with time as a dimension, because each method catches what the others miss.
5. **Security and provenance are load-bearing.** ACLs filter before ranking; citations verify before display; every answer leaves an immutable trail; abstention is a graded skill.
6. **Trust is re-earned nightly.** Evaluation gates, reconciliation counts, hallucination sampling, restore drills — the system proves it still works after every day's changes, or it stops and says so.

A corpus of three million documents, one honest database, and a system designed to be *caught* being wrong before a user ever is. That's the design.

Appendix A: Requirements Reference

A.1 Functional

ID	Requirement
F1	Ingest documents from multiple enterprise sources (file shares, SharePoint, email archives, document management systems) in PDF, DOCX, PPTX, XLSX, TXT, HTML, Markdown, and common scanned image formats.
F2	Handle scanned documents via OCR, with extraction confidence scored and attached at the chunk/page level.
F3	Preserve document hierarchy: section headers, page and slide numbers, table schemas, footnotes, and speaker notes.
F4	Detect and handle exact duplicates, near-duplicates, and superseded versions with alias mapping.
F5	Support hybrid retrieval — exact/lexical match (tickers, ISINs, clause numbers, dates, party names) and semantic match (conceptual questions, thematic synthesis, paraphrase).
F6	Filter by rich metadata: source, business unit, document type, date range, classification, and language.
F7	Return answers with verifiable citations at the page / slide / section / table level, deterministically linked to the exact document version used.
F8	Abstain or flag uncertainty when evidence is weak, contradictory, or superseded.
F9	Expose retrieval as an independent, secured REST/gRPC service consumable by the agent and other internal systems.
F10	Re-index automatically upon source modification; tombstone deleted content promptly.

ID	Requirement
F11	Provide a human-browsable, normalized Markdown corpus for domain expert QA, curation, and compliance audits.
F12	Maintain a typed inter-document link graph (supersession, amendment, citation, exhibit, shared entities) and an entity index in PostgreSQL, populated deterministically at ingestion and usable for retrieval-time context expansion.
F13	Render the corpus as an interconnected wiki: Markdown files with YAML frontmatter and <code>[[wikilinks]]</code> (Obsidian-compatible), generated from the database, human-navigable and grep-friendly, scoped to authorized audiences.
F14	Support point-in-time ("as-of") retrieval via an optional <code>as_of_date</code> parameter, with historical answers explicitly flagged.

A.2 Security, Compliance, and Governance

ID	Requirement
S1	Access control strictly mirrors source-system entitlements. ACL filters are applied pre-ranking at the database layer; unauthorized users can never retrieve or observe chunks from restricted documents.
S2	Information barriers (ethical walls, client/deal segregation) enforced at the tenant / collection partition level.
S3	Automated classification tags at ingestion: PII, MNPI, client-confidential, regulatory, and retention category.
S4	Full audit log per query: user identity, evaluated roles/groups, applied filters, retrieved chunk IDs, ranking scores, model/prompt versions, generated response, and rendered citations.
S5	Complete data residency: originals, derivatives, vector indices, and audit logs remain within the firm's approved security boundary. Models are self-hosted or private enterprise endpoints.
S6	Release v1 is strictly read-only : no execution of financial transactions, external communications, or automated decisions.
S7	Prompts, retrieved contexts, and outputs are archived as immutable business records; closed audit-log partitions are exported to write-once storage.

A.3 Non-Functional

ID	Requirement
N1	Corpus Scale: Low single-digit millions of source documents; 20–30 million indexed chunks after deduplication.
N2	Query Latency: p95 retrieval under 1.5 s (excluding LLM generation).
N3	Ingestion: Backfill in weeks; incremental sync in minutes.
N4	Operational Simplicity: Zero multi-cluster maintenance overhead for v1.
N5	Reliability: Transactional, observable, crash-resilient job processing with automatic retries.
N6	Measurable Quality: Recall@20, citation accuracy, groundedness, and permission-leak rate continuously evaluated against a golden dataset.
N7	Portability: Chunk indices are disposable and 100% reconstructible from stored derivatives without re-parsing raw files.
N8	Storage Platform: Originals and derivatives on mounted block/file volumes inside the firm's network — no object-store dependency.
N9	Backup & Recovery: RPO $\leq$ 1 hour, RTO $\leq$ 8 hours; immutable backups inside the security boundary; automated restore tests at least monthly.

ID	Requirement
N10	Lifecycle & Continuous Evaluation: Content changes visible within minutes; ACL revocations within 5 minutes; nightly evaluation gate verifies quality, completeness, and hallucination level after each day's updates and freezes ingestion on regression.

Appendix B: Technology Choices

Layer	Selected Technology	Why
Originals & derivatives	Mounted volumes (EBS/NFS or NAS/SAN), XFS + LVM, content-addressed and write-once	In-boundary, snapshot-consistent by construction, POSIX-fast for parsers
Parsing	Docling (heavy path) + PyMuPDF (fast path)	Layout/table fidelity with OCR confidence; 10× faster path for born-digital files
Database & indexing	PostgreSQL 16+ with pgvector (HNSW, halfvec) and pg_search (BM25)	One engine for ACL joins, keyword search, vector search, queue, and audit
Document graph & wiki	Plain PostgreSQL edge/entity tables; wiki rendered as Markdown + frontmatter + wikilinks (Obsidian-compatible)	Structure stays in the system of record; wiki is a disposable projection
Job queue	PostgreSQL SKIP LOCKED	Transactional, broker-free
Embeddings	Self-hosted open models (BGE / E5 / Nomic class) on local GPUs	Data boundary; no per-token fees across 10B+ tokens
Reranker	BGE-Reranker-Large cross-encoder; LLM rerank only as evaluated escalation	Precision on multi-hop clauses within the latency budget
Retrieval API	Python FastAPI / asyncpg	Lightweight, typed, decoupled from agents
LLM inference	Private enterprise endpoint / self-hosted vLLM	Inside the VPC perimeter
Backup & recovery	pgBackRest PITR + standby; locked cross-AZ snapshots; WORM audit exports	One-hour RPO; ransomware-resistant; restore path proven by drills

Appendix C: Risk Register (Condensed)

Risk	Mitigation
Permission leakage (incl. via graph links or wiki)	Pre-ranking SQL ACL joins everywhere including graph hops; wiki rendered only in curator enclave; adversarial leak tests in CI; 0.00% tolerance
Missing specific clauses	Adaptive routing lets identifier queries bypass summary filtering
Stale or wrong-period answers	Validity windows, as-of filtering, temporal/currency ranking boosts, supersession flags, as-of golden cases
Hallucinated citations	Middleware validates every citation token against retrieved chunks; unanswerable-set testing; nightly judge sampling
Silent ingestion failure / stale index	Nightly three-layer reconciliation; index-lag metric; self-retrieval probes
Quality regression after updates	Nightly golden-suite gate vs. 7-day baseline; page + ingestion freeze on trip
Document volume loss/corruption	Locked cross-AZ snapshots; write-once discipline; hash scrub

Risk	Mitigation
PostgreSQL loss / logical corruption	Continuous WAL archiving + PITR; standby for failover; monthly restore tests
Audit record loss	Monthly partitions exported nightly to WORM with hash manifests; records-policy-only deletion
Backup destruction (ransomware / rogue admin)	Snapshot lock, separate backup account, WORM archive, scrub-verified clean-point restore
Untested restore path	Monthly automated restore tests page on failure; quarterly timed DR drills reported to Compliance/BC